

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Pseudocode Writing Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO5 – Naturalization (BL5, BL6)	Assessment:	Assessment Tool 2 – Pseudocode Writing Task
Weightage:	30% of CIA	Total Marks:	100
CO5 Statement:	<i>Analyze computational problems using backtracking techniques and complexity classes such as P, NP, NP-Hard, and NP-Complete for combinatorial problem solving.</i>		
Student Name:	D.HEMASREE	Reg. No.:	192524114
Section / Batch:	2025	Date:	

Instructions to Students

- Answer the following question with proper pseudocode and logical explanation.
- Clearly specify the algorithmic approach and complexity class wherever applicable.
- Use proper asymptotic notation (Big-O, Big-Θ, Big-Ω) for complexity analysis.
- Include state-space representation, recursion steps, and pruning conditions in backtracking problems.
- Neat presentation and step-by-step justification are mandatory.

Question Type	Focus Area	Questions
Pseudocode Writing Task	Analyze combinatorial problems using backtracking and complexity classes such as P, NP, NP-Hard, and NP-Complete.	Q1

CODE:

import copy

def is_safe(grid, row, col, num):

"""Check if it is legal to place num in grid[row][col]."""

Check row

for x in range(9):

if grid[row][x] == num:

return False

```

# Check column
for x in range(9):
    if grid[x][col] == num:
        return False

# Check 3x3 subgrid
start_row = row - row % 3
start_col = col - col % 3

for i in range(3):
    for j in range(3):
        if grid[i + start_row][j + start_col] == num:
            return False

return True

```

```

def find_empty_location(grid):
    """Finding an empty cell (marked as 0). Returns (row, col) or None."""
    for i in range(9):
        for j in range(9):
            if grid[i][j] == 0:
                return (i, j)
    return None

```

```

def solve_sudoku(grid, all_solutions):
    """Uses backtracking to find all valid solutions."""
    loc = find_empty_location(grid)

    if not loc:
        all_solutions.append(copy.deepcopy(grid))
        return True

    row, col = loc

    for num in range(1, 10):
        if is_safe(grid, row, col, num):
            grid[row][col] = num

            solve_sudoku(grid, all_solutions)

```

```
grid[row][col] = 0
```

```
return False
```

```
def print_grid(grid):
```

```
    """Utility to print the grid in a readable format."""
```

```
    for i in range(9):
```

```
        if i % 3 == 0 and i != 0:
```

```
            print("-----+-----+-----")
```

```
        for j in range(9):
```

```
            if j % 3 == 0 and j != 0:
```

```
                print("|", end=" ")
```

```
            print(grid[i][j], end=" ")
```

```
        print()
```

```
def main():
```

```
    input_grid = [
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
        [0, 3, 0, 0, 7, 0, 0, 0, 0],
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 3],
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 1],
```

```
        [0, 0, 0, 0, 0, 2, 0, 0, 0],
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 0],
```

```
        [0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
    ]
```

```
    print("=== Sudoku Solver (Find All Solutions) ===")
```

```
    all_solutions = []
```

```
    solve_sudoku(input_grid, all_solutions)
```

```
    if not all_solutions:
```

```
        print("\nNo solutions found.")
```

```
    else:
```

```
        print(f"\nFound {len(all_solutions)} solution(s):")
```

```
        for idx, sol in enumerate(all_solutions):
```

```
print(f"\nSolution {idx + 1}:")
print_grid(sol)
```

```
if __name__ == "__main__":
    main()
```

OUTPUT:

ASSESSMENT- CO5-AT2-Pseudocode Writing Task (Algorithm Design)

← Back

Language: Python C C++ Java

QUESTIONS

Q1

Sudoku Solver for Finding All Possible Solutions

100 marks

1/1 answered

Q1 Sudoku Solver for Finding All Possible Solutions

100 marks

Write a pseudocode using Backtracking to generate all valid solutions for a given Sudoku puzzle. Clearly define inputs (partially filled Sudoku grid) and output (all possible completed grids). Design an algorithm that recursively explores every legal value for each empty cell. Use loops and recursive calls effectively to continue the search after each valid placement. Apply pruning whenever row, column, or subgrid rules are violated. Present the pseudocode in a neat and logically structured format. Ensure completeness by exploring all valid possibilities without missing any solution.

Your Code

```
1: copy
2: safe(grid, row, col, num):
3: """Checks if it is legal to place num in grid[row][col]."""
4: for x in range(9):
5:     if grid[row][x] == num:
6:         return False
7: for x in range(9):
8:     if grid[x][col] == num:
9:         return False
10: art_row = row - row % 3
11: art_col = col - col % 3
12: for i in range(3):
13:     for j in range(3):
14:         if grid[i + start_row][j + start_col] == num:
15:             return False
16: return True
17: find_empty_location(grid):
18: """Finds an empty cell (marked as 0). Returns (row, col) or None."""
19: for i in range(9):
20:     for j in range(9):
21:         if grid[i][j] == 0:
22:             return i, j
```

Input

stdin input

Output

```
--- Sudoku Solver (Find All
Solutions) ---

Found 1 solution(s):

Solution 1:
```

SECTION A – Pseudocode Writing Task

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Marks Evaluation:

			Pseudocode Structure	10%	
Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	
		Completeness			20%
Problem Understanding	20%				
			Total Marks (100):		
Algorithm Design	30%				
Use of Control Structures	20%				

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____

Date: _____	Date: _____	Date: _____
-------------	-------------	-------------